UNIVERSITÉ JOSEPH KI-ZERBO
****
UNITÉ DE FORMATION ET DE RECHERCHE
EN SCIENCES EXACTES ET APPLIQUÉES
****
DÉPARTEMENT INFORMATIQUE
(Master 1)
Année académique 2025-2026

Rapport : Services Web

TP – Conception et utilisation d'un service REST

Enseignant :
Dr. SOMBDA

Membres du groupe :
GNISSIEN Migouel
TAPSOBA Sandrine Pelagie

Table des matières

Liste des figures **3**

Introduction **4**

1 Partie 1 — Modélisation REST **5**
1.1 Identification des ressources 5
1.2 Définition des URI et Verbes HTTP 6

2 Partie 2 — Consommation de l'API **8**
2.1 Création d'un cours (POST) 8
2.2 Consultation (GET) 9

3 Partie 3 — Mise à jour : PUT vs PATCH **10**
3.1 PUT (Remplacement complet) 10
3.2 PATCH (Modification partielle) 10

4 Partie 4 — Codes HTTP et gestion des erreurs **12**
4.1 Exemple d'erreur 404 12

5 Partie 5 — Bonus (Niveau Master) **13**
5.1 Validation métier : volumeHoraire > 0 13
5.2 Pagination des ressources 13

6 Technologies Utilisées **15**

Conclusion **18**

Table des figures

1 Requête POST réussie sous Postman 8
2 Récupération de la collection des cours (GET) 9
3 Résultat de la mise à jour complète (PUT) 10
4 Résultat de la modification partielle (PATCH) 11
5 Capture d'écran de l'erreur 404 rencontrée 12
6 Échec de la validation métier 13
7 Visualisation de la pagination sous Postman 14
8 Logo du langage Python 15
9 Logo du Framework Django REST 15
10 Logo de PostgreSQL 16
11 Logo de l'outil Postman 17

Introduction

Le développement moderne des applications distribuées repose en grande partie sur l'échange de données entre systèmes hétérogènes. Dans ce contexte, les services web et l'architecture REST se sont imposés comme des standards incontournables grâce à leur simplicité et leur scalabilité. Ce rapport présente les travaux réalisés dans le cadre du cours de Master 1. L'étude se concentre sur la mise en place d'une architecture conforme aux bonnes pratiques, notamment par l'utilisation d'URI sémantiques et la manipulation rigoureuse des verbes HTTP fondamentaux. Nous abordons ainsi les opérations de création, de lecture et de suppression, tout en approfondissant la gestion des modifications via les méthodes PUT et PATCH, ainsi que des fonctionnalités avancées de validation et de pagination.

1 Partie 1 — Modélisation REST

1.1 Identification des ressources

L'architecture du système repose sur une modélisation orientée ressources, où le **Cours** constitue l'entité pivot autour de laquelle gravitent les interactions des autres composants du domaine. Les ressources identifiées pour ce système sont les suivantes :

- **Cours**
- **Étudiant**
- **Enseignant**
- **Inscription**

1.2 Définition des URI et Verbes HTTP

Action prévue	Méthode	Point de terminaison (URI)
Cours		
Lister les cours	GET	/api/cours
Lire un cours	GET	/api/cours/{id_cours}
Créer un cours	POST	/api/cours
Modifier un cours	PUT	/api/cours/{id_cours}
Modifier partiellement	PATCH	/api/cours/{id_cours}
Supprimer un cours	DELETE	/api/cours/{id_cours}
Étudiants		
Lister les étudiants	GET	/api/etudiants
Lire un étudiant	GET	/api/etudiants/{id_etudiant}
Créer un étudiant	POST	/api/etudiants
Modifier un étudiant	PUT	/api/etudiants/{id_etudiant}
Supprimer un étudiant	DELETE	/api/etudiants/{id_etudiant}
Enseignants		
Lister les enseignants	GET	/api/enseignants
Lire un enseignant	GET	/api/enseignants/{id_enseignant}
Créer un enseignant	POST	/api/enseignants
Modifier un enseignant	PUT	/api/enseignants/{id_enseignant}
Supprimer un enseignant	DELETE	/api/enseignants/{id_enseignant}
Inscriptions		
Lister les inscriptions	GET	/api/inscriptions
Inscrire un étudiant	POST	/api/inscriptions
Supprimer une inscription	DELETE	/api/inscriptions/{id_inscription}
Relations		
Cours d'un étudiant	GET	/api/etudiants/{id_etudiant}/cours

Justification des choix REST

- ✓ **Légèreté et Performance :** Contrairement à SOAP qui utilise du XML lourd, REST privilégie le JSON, réduisant la taille des messages et accélérant le traitement, un atout majeur pour les applications mobiles.
- ✓ **Interopérabilité :** En se basant sur le protocole HTTP standard, REST permet à des systèmes totalement différents (Web, Mobile, IoT) de communiquer sans protocoles complexes supplémentaires.
- ✓ **Scalabilité :** L'architecture est *stateless* (sans état). Le serveur ne stocke pas de session client, ce qui facilite la montée en charge et la gestion de milliers de requêtes simultanées.
- ✓ **Simplicité de mise en œuvre :** L'utilisation des verbes HTTP (GET, POST, etc.) rend l'interface intuitive pour les développeurs par rapport aux architectures RPC.

2 Partie 2 — Consommation de l'API

Note sur la sécurité des identifiants :
Pour la suite des requêtes présentées, nous allons utiliser des **ID complexes** (mélange de caractères et de chiffres). Contrairement aux identifiants auto-incrémentés simples (1, 2, 3), l'utilisation de chaînes alphanumériques complexes permet de sécuriser l'API en évitant les attaques par énumération de ressources.

2.1 Création d'un cours (POST)

```
POST /api/cours
{
  "titre": "Architecture Logicielle",
  "enseignant": "Dr. Somda",
  "volumeHoraire": 24,
  "actif": true
}
```

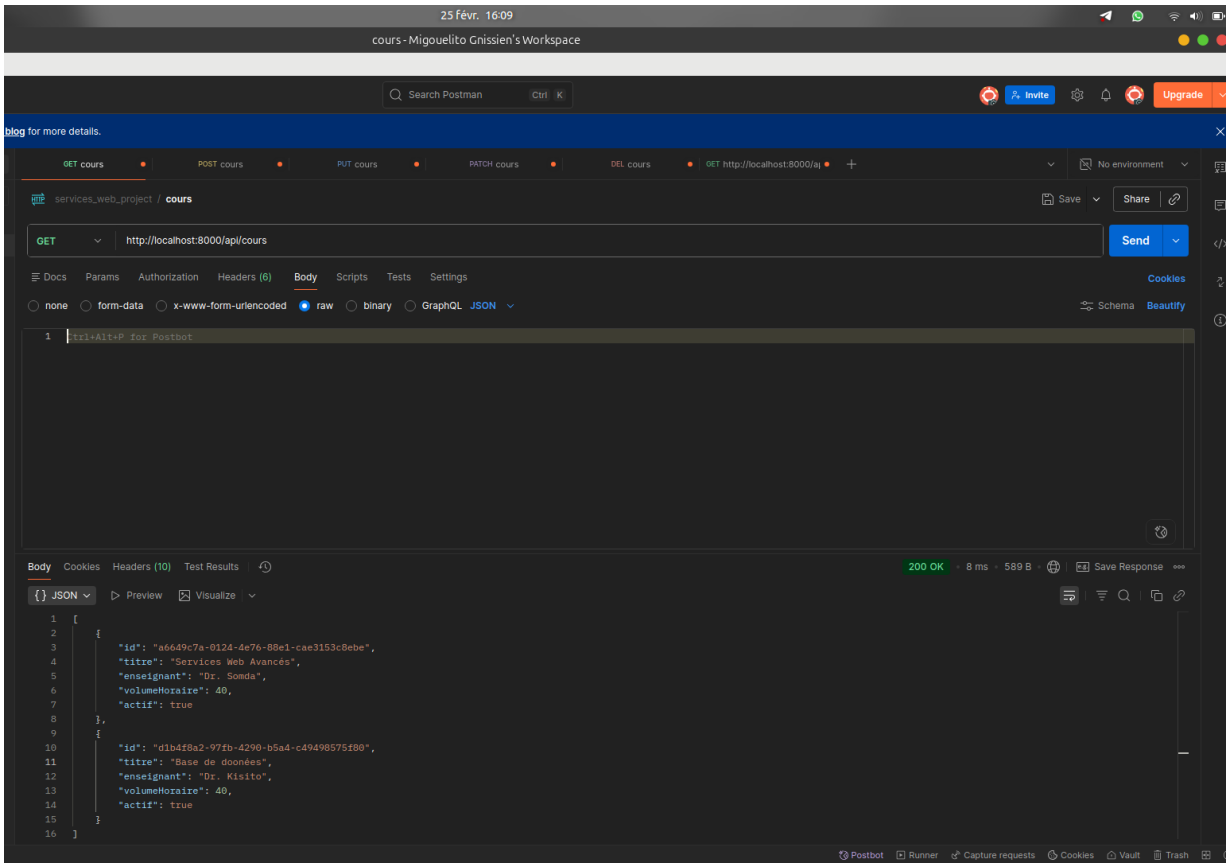

FIGURE 1 – Requête POST réussie sous Postman

2.2 Consultation (GET)

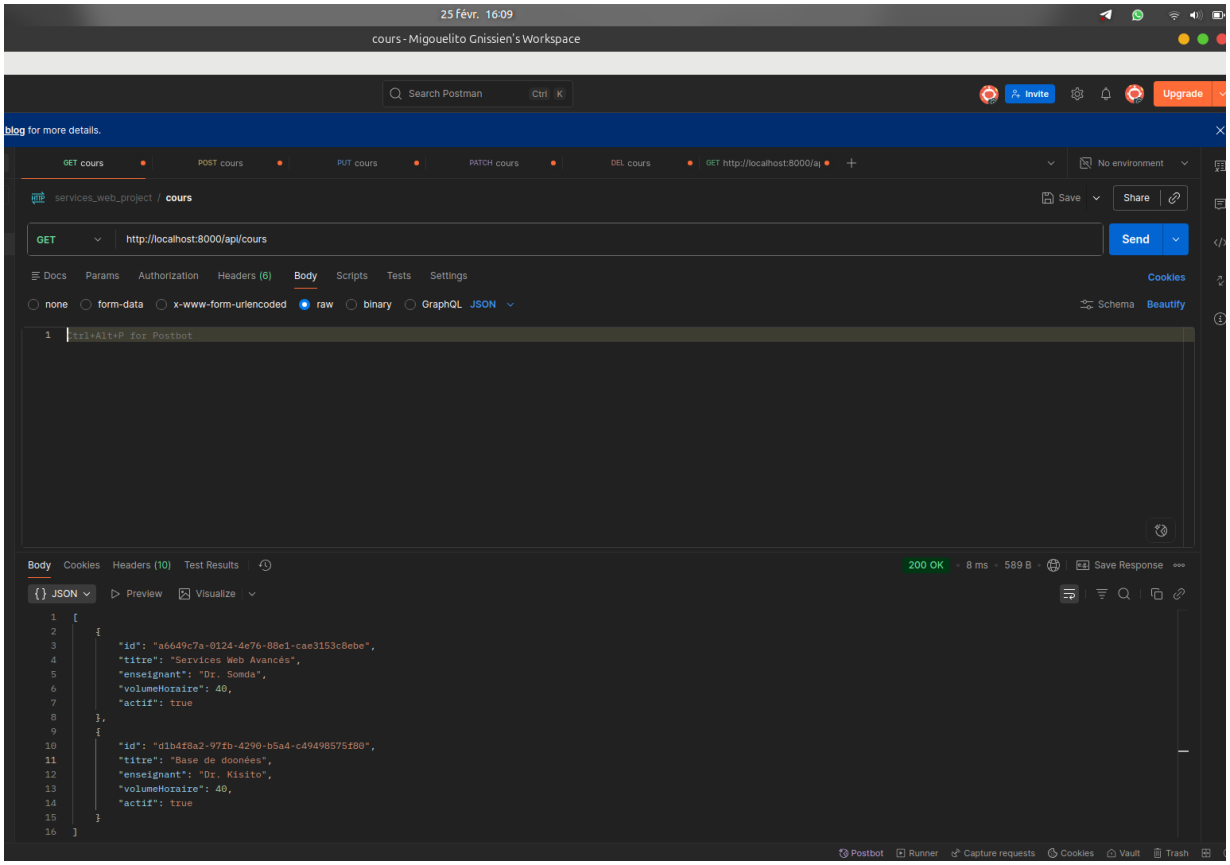

FIGURE 2 – Récupération de la collection des cours (GET)

3 Partie 3 — Mise à jour : PUT vs PATCH

3.1 PUT (Remplacement complet)

```
PUT /api/cours/1
{
  "id": "7f8b9c2a1-e4d5-4a2b-8c7d-9e0f1a2b3c4d",
  "titre": "Services Web Avancés",
  "enseignant": "Dr. Somda",
  "volumeHoraire": 36,
  "actif": true
}
```

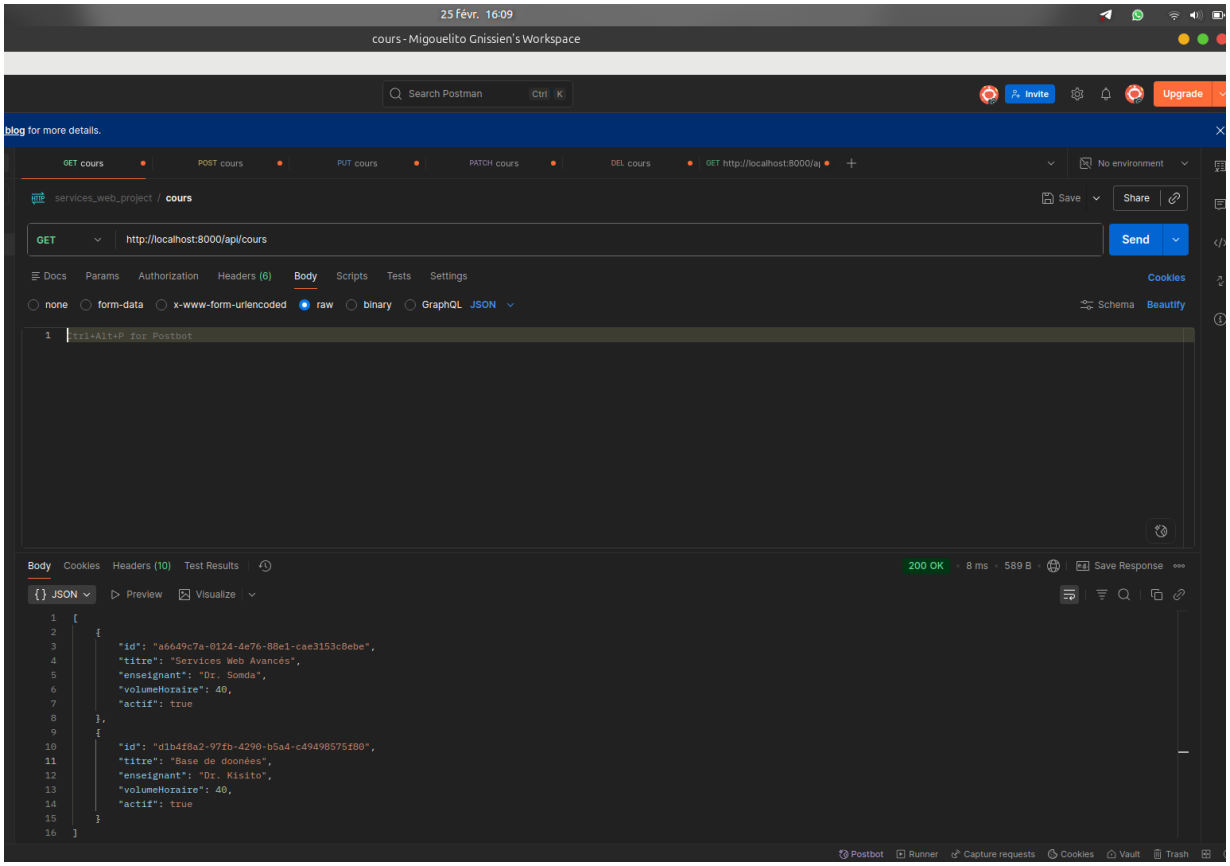

FIGURE 3 – Résultat de la mise à jour complète (PUT)

3.2 PATCH (Modification partielle)

```
PATCH /api/cours/1
[
  { "op": "replace", "path": "/volumeHoraire", "value":
      40 }
]
```

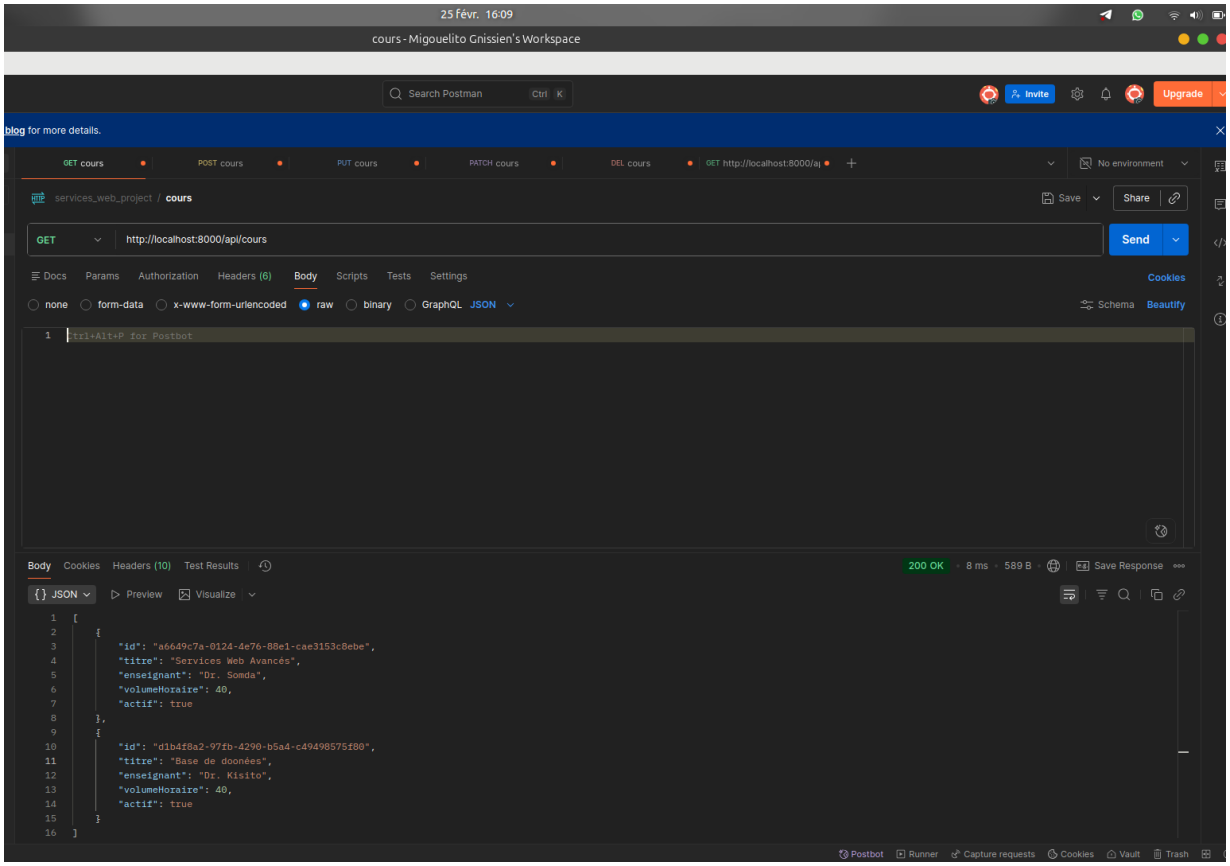

Figure 4 – Résultat de la modification partielle (PATCH)

4 Partie 4 — Codes HTTP et gestion des erreurs

4.1 Exemple d'erreur 404

```
{
  "error": "Cours non trouvé",
  "timestamp": "2026-02-20T10:30:00"
}
```

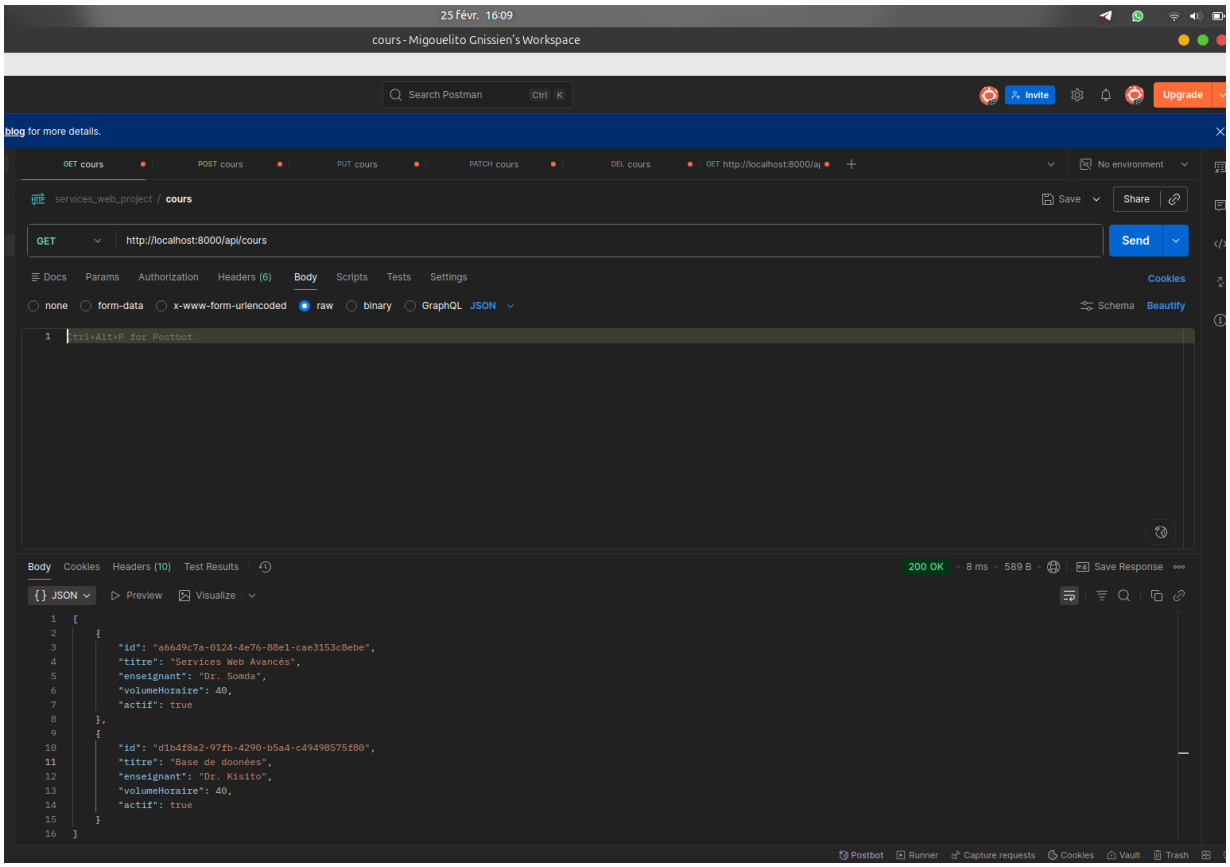

FIGURE 5 – Capture d'écran de l'erreur 404 rencontrée

5 Partie 5 — Bonus (Niveau Master)

5.1 Validation métier : volumeHoraire > 0

Une contrainte a été ajoutée pour empêcher l'insertion de volumes horaires incohérents.

```
POST /api/cours
{
  "titre": "Test Erreur",
  "volumeHoraire": -10
}
```

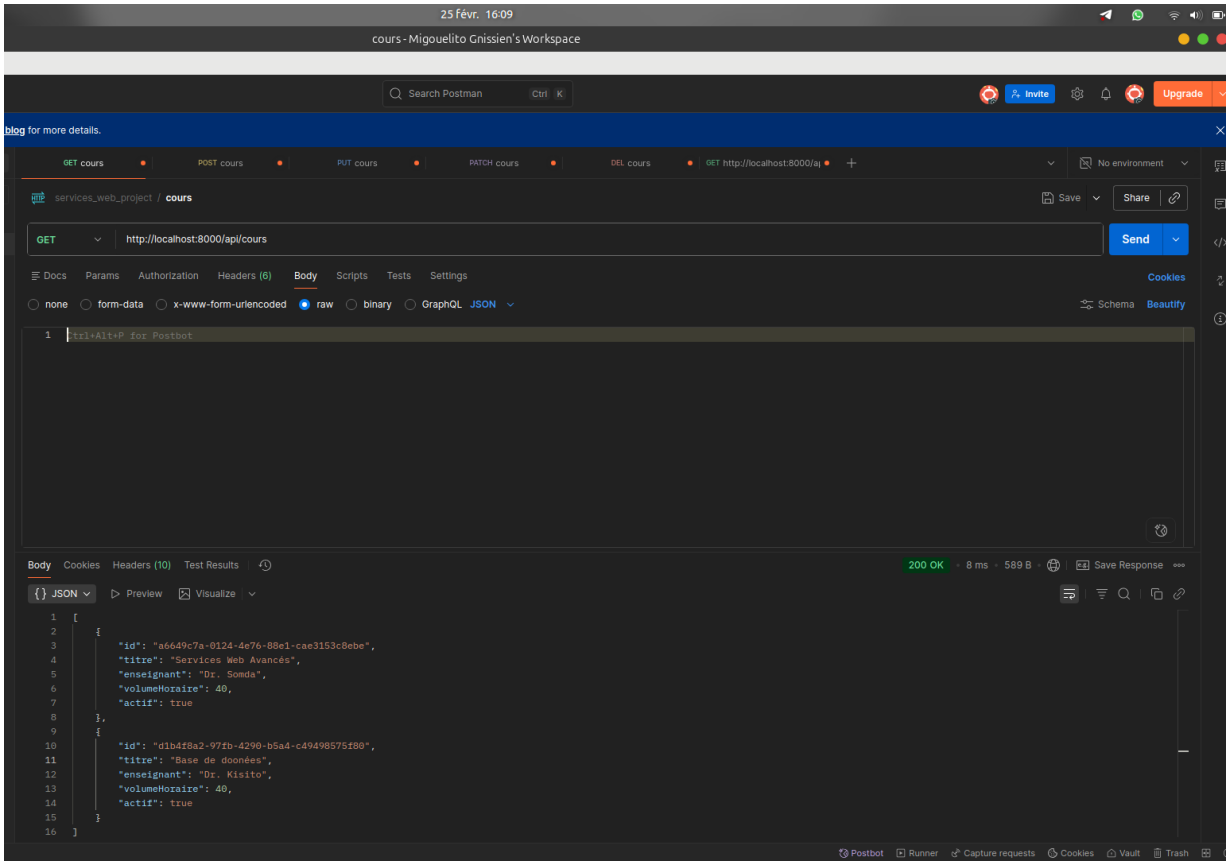

FIGURE 6 – Échec de la validation métier

5.2 Pagination des ressources

Pour optimiser les performances, nous avons implémenté la pagination via les paramètres `page` et `size`.

```
GET /api/cours?page=0&size=5
```

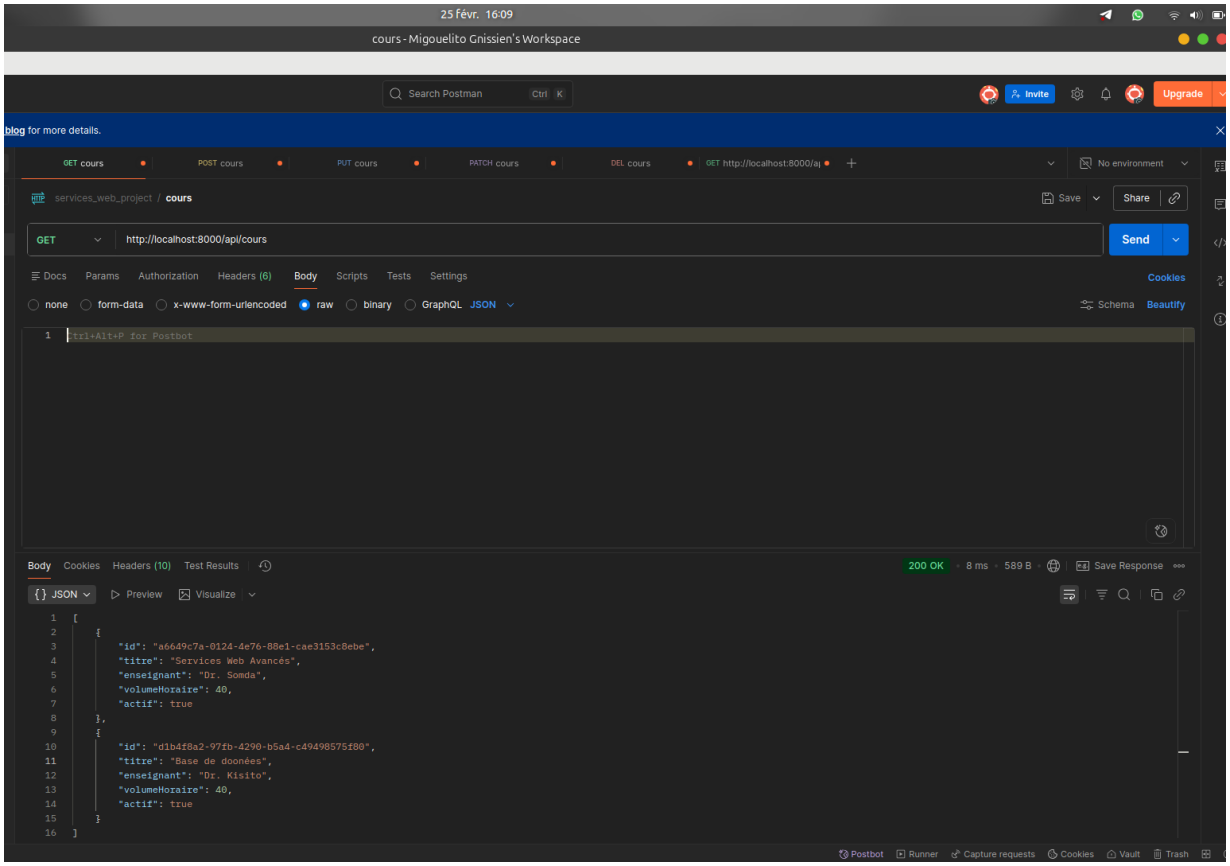

FIGURE 7 – Visualisation de la pagination sous Postman

6 Technologies Utilisées

La mise en œuvre de ce service web repose sur des outils modernes garantissant rapidité de développement et robustesse des tests.

Langage Python : Utilisé pour sa syntaxe concise et sa large bibliothèque standard, idéal pour le traitement de données et la logique métier.

Figure 8 – Logo du langage Python

Framework Django : Nous avons exploité Django REST Framework pour construire l'API. Son architecture robuste permet une séparation claire entre les modèles de données et les contrôleurs.

Figure 9 – Logo du Framework Django REST

Base de données PostgreSQL : Système de gestion de base de données relationnelle puissant et open-source, utilisé pour assurer la persistance et l'intégrité des données du service.

Figure 10 – Logo de PostgreSQL

Postman : Cet outil a été au cœur de notre processus de développement pour simuler les requêtes clientes, inspecter les réponses JSON et valider les codes de statut HTTP.

Figure 11 – Logo de l'outil Postman

L'utilisation de ces technologies garantit une API performante et conforme aux standards industriels.

Conclusion

En conclusion, ce travail pratique nous a permis de consolider nos connaissances sur l'architecture REST et son application concrète. La manipulation des verbes HTTP, l'usage de PUT et PATCH, ainsi que l'ajout de la validation métier et de la pagination, démontrent les exigences de robustesse d'une API de niveau Master. Ces compétences sont essentielles pour assurer l'interopérabilité et la performance des systèmes distribués modernes.

MERCI